

Sesión 19: Funciones agregadas, GROUP BY y HAVING

1. Objetivos de la sesión

- Entender qué son las funciones agregadas y cuándo se usan.
- Diferenciar consultas normales de consultas agrupadas.
- Utilizar correctamente GROUP BY.
- Comprender la diferencia entre WHERE y HAVING.
- Combinar WHERE, GROUP BY y HAVING en una misma consulta.
- Evitar los errores más comunes en consultas agrupadas.

2. Introducción

Hasta ahora:

- Cada fila del resultado representaba **un registro**.
- Trabajábamos con datos “tal cual están en la tabla”.

A partir de ahora:

- Vamos a trabajar con **conjuntos de registros**.
- El resultado ya no representa filas individuales, sino **resúmenes**.

Ejemplo:

- Antes: muéstrame todas las incidencias.
- Ahora: dime cuántas incidencias hay por estado.

3. Funciones agregadas

Las funciones agregadas permiten realizar **cálculos sobre varias filas** y devolver **un único valor** por grupo (o por toda la tabla).

Funciones agregadas principales

- COUNT(): cuenta filas
- SUM(): suma valores numéricos
- AVG(): media
- MAX(): valor máximo
- MIN(): valor mínimo

3.1 COUNT()

Cuenta registros.

```
SELECT COUNT(*) FROM usuarios;
```

Cuenta todas las filas.

```
SELECT COUNT(email) FROM usuarios;
```

Cuenta solo las filas donde email **no es NULL**.

Ejemplo:

```
SELECT COUNT(*) FROM incidencias WHERE estado = 'abierta';
```

3.2 SUM() y AVG()

Solo se usan con campos numéricos.

```
SELECT SUM(precio) FROM reparaciones;
```

```
SELECT AVG(precio) FROM reparaciones;
```

Ejemplo:

```
SELECT AVG(precio)
FROM reparaciones
WHERE vehiculo_id = 3;
```

3.3 MAX() y MIN()

```
SELECT MAX(precio) FROM reparaciones;
```

```
SELECT MIN(fecha) FROM incidencias;
```

Ejemplo:

```
SELECT MAX(fecha)
FROM incidencias
WHERE estado = 'resuelta';
```

4. Agrupación de datos: GROUP BY

GROUP BY permite **agrupar filas que comparten un mismo valor** y aplicar funciones agregadas a cada grupo.

4.1 Ejemplo básico

Número de incidencias por estado:

```
SELECT estado, COUNT(*)  
FROM incidencias  
GROUP BY estado;
```

Resultado posible:

- abierta : 10
- en_proceso: 5
- resuelta: 18

4.2 Regla fundamental del GROUP BY

Todo campo que aparece en el SELECT y **no es una función agregada**, debe aparecer en el GROUP BY.

Incorrecto:

```
SELECT estado, fecha, COUNT(*)  
FROM incidencias  
GROUP BY estado;
```

1. Qué le estás pidiendo a la base de datos

Le dices:

- Agrupa las incidencias por estado.
- Para cada grupo, muéstrame:
 - el estado
 - la fecha
 - el número de incidencias (COUNT(*))

2. El problema real: fecha no tiene un único valor

Supón que la tabla tiene estos datos:

estado	fecha
abierta	2024-01-01
abierta	2024-01-03
abierta	2024-01-10

Cuando haces:

GROUP BY estado

todas esas filas pasan a ser **un solo grupo**:

- estado = abierta
- COUNT(*) = 3

Pero ahora viene el problema:

¿Qué fecha debería mostrar SQL?

- 2024-01-01
- 2024-01-03
- 2024-01-10

No hay una única fecha correcta, porque dentro del grupo hay **varias fechas distintas**.

3. Por eso SQL la considera incorrecta

SQL exige que **cada columna del SELECT tenga un valor único por grupo**.

Correcto:

```
SELECT estado, COUNT(*)  
FROM incidencias  
GROUP BY estado;
```

Cuando agrupas por estado, pierdes el detalle de cada fila. No puedes pedir una fecha concreta si hay varias dentro del grupo.

4.3 Más ejemplos de GROUP BY

Incidencias por prioridad:

```
SELECT prioridad, COUNT(*)  
FROM incidencias  
GROUP BY prioridad;
```

Precio medio de reparaciones por vehículo:

```
SELECT vehiculo_id, AVG(precio)  
FROM reparaciones  
GROUP BY vehiculo_id;
```

Incidencias por estado y prioridad:

```
SELECT estado, prioridad, COUNT(*)  
FROM incidencias  
GROUP BY estado, prioridad;
```

5. WHERE en consultas agrupadas

WHERE **filtra filas antes de agrupar.**

Ejemplo:

```
SELECT estado, COUNT(*)  
FROM incidencias  
WHERE prioridad = 'alta'  
GROUP BY estado;
```

Aquí:

- Primero se eliminan las incidencias que no son prioridad alta.
- Luego se agrupan por estado.

6. HAVING: filtrado de grupos

HAVING se usa para **filtrar resultados agregados**.

Diferencia clave

- WHERE: trabaja con filas
- HAVING: trabaja con grupos

6.1 Ejemplo básico de HAVING

Estados con más de 5 incidencias:

```
SELECT estado, COUNT(*) AS total
FROM incidencias
GROUP BY estado
HAVING COUNT(*) > 5;
```

6.2 Error típico

```
SELECT estado, COUNT(*)
FROM incidencias
GROUP BY estado
WHERE COUNT(*) > 5;
```

Esto falla porque COUNT(*) aún no existe cuando se ejecuta WHERE.

6.3 HAVING con alias

```
SELECT estado, COUNT(*) AS total
FROM incidencias
GROUP BY estado
HAVING total > 5;
```

7. WHERE + GROUP BY + HAVING juntos

Ejemplo completo:

Mostrar los estados con más de 3 incidencias **de prioridad alta**:

```
SELECT estado, COUNT(*) AS total  
FROM incidencias  
WHERE prioridad = 'alta'  
GROUP BY estado  
HAVING COUNT(*) > 3;
```

Orden correcto:

1. WHERE filtra filas
2. GROUP BY agrupa
3. Funciones agregadas calculan
4. HAVING filtra grupos
5. SELECT muestra resultados

8. Errores comunes

- Usar WHERE con funciones agregadas.
- Olvidar campos en el GROUP BY.
- Creer que HAVING sustituye a WHERE.
- Esperar filas individuales en una consulta agrupada.
- Mezclar campos sin sentido en el SELECT.